

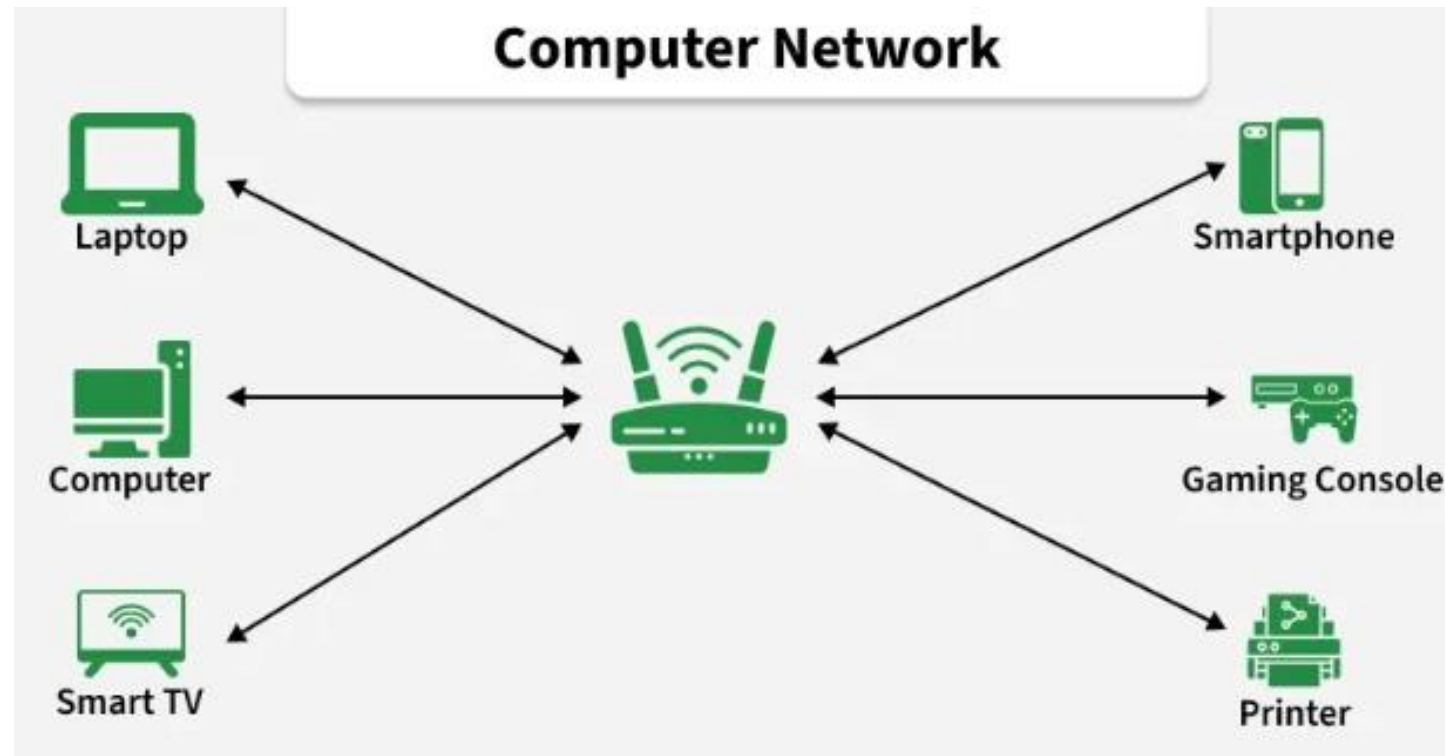


CSE357: COMBINATORIAL STUDIES

Unit II: Computer Networking Basics

- Foundations of Computer Networks
- Network Topologies and Transmission Medium
- Understanding OSI and TCP/IP Models
- Subnetting and Routing
- Key Protocols: HTTP, SMTP, POP/IMAP, FTP, DNS, DHCP

A computer network is a system of interconnected devices, such as computers, servers, smartphones, and printers, that communicate to exchange data. They communicate using wired or wireless connections and can range from small home networks to the global Internet.



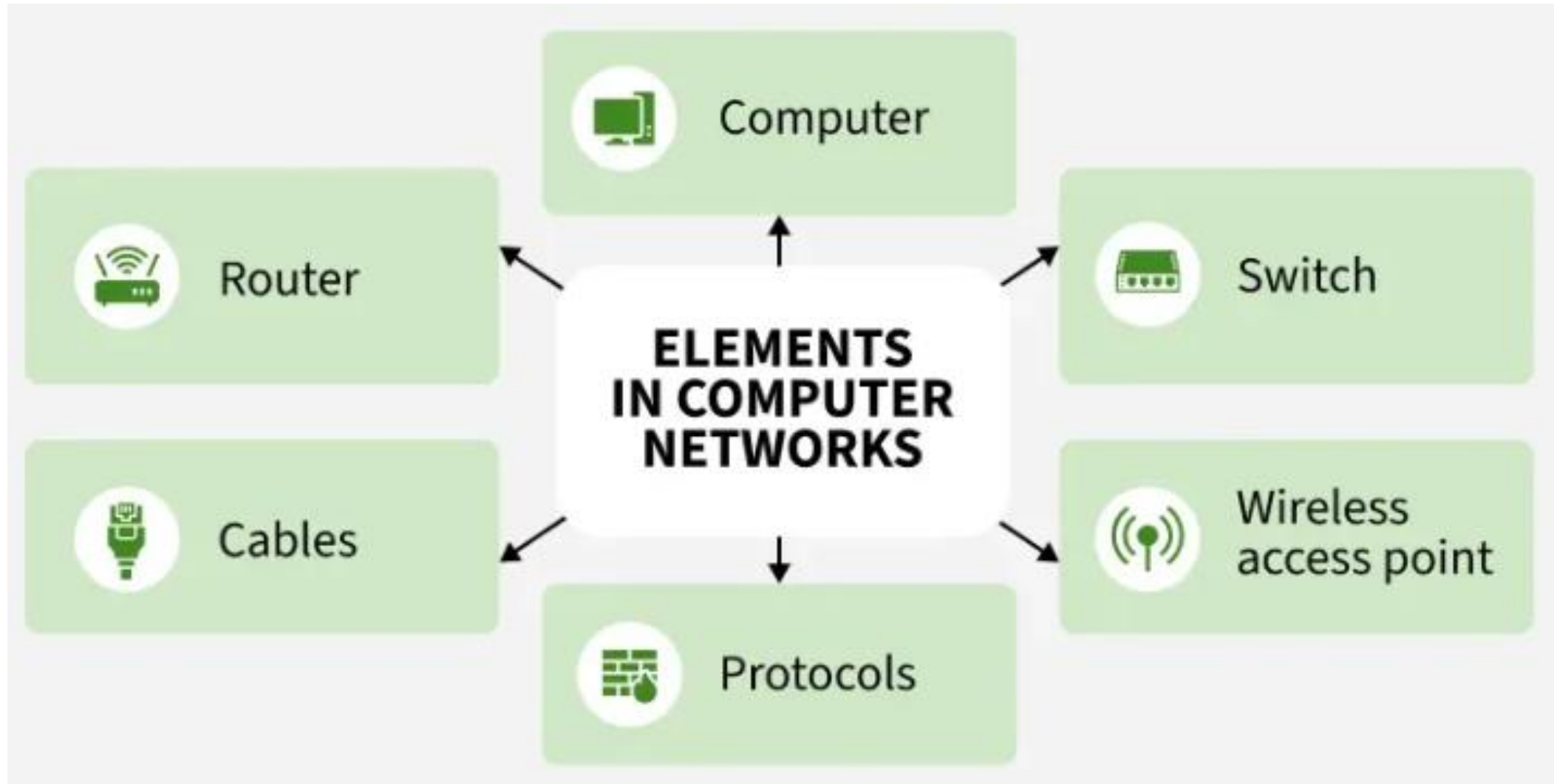
Why Do We Need Computer Networks?

- **Resource Sharing:** Allows sharing of hardware like printers, scanners, and storage devices, reducing cost.
- **Data Sharing:** Enables users to share files, applications, and databases easily.
- **Communication:** Supports email, video conferencing, instant messaging, and web access.
- **Data Management:** Helps in storing, securing, and backing up data efficiently.
- **Remote Access:** Allows users to access systems and cloud services from anywhere.
- **Better Collaboration:** Improves teamwork and information exchange among users.

Elements of Computer Network

- A computer network is a system of interconnected computers and devices that communicate with each other to share data and resources.
- Whether it's the internet, your school's Wi-Fi, or a company's internal system, all networks are built on some basic elements.

Computer Network Elements



Types of Computer Networks are classified based on several factors:

1. Geographical area
2. Transmission medium
3. Ownership and access control

Classification Based on Geographical Area



**Personal
Area
Network
(PAN)**



**Local Area
Network
(LAN)**



**Campus
Area
Network
(CAN)**



**Metropolitan
Area
Network
(MAN)**



**Wide Area
Network
(WAN)**

Classification Based on Transmission Medium



**Broadcast
Network**



**Peer-to-Peer
Network**

Classification Based on Ownership and Access Control



**Private
Network**



**Public
Network**



**Hybrid
Network**

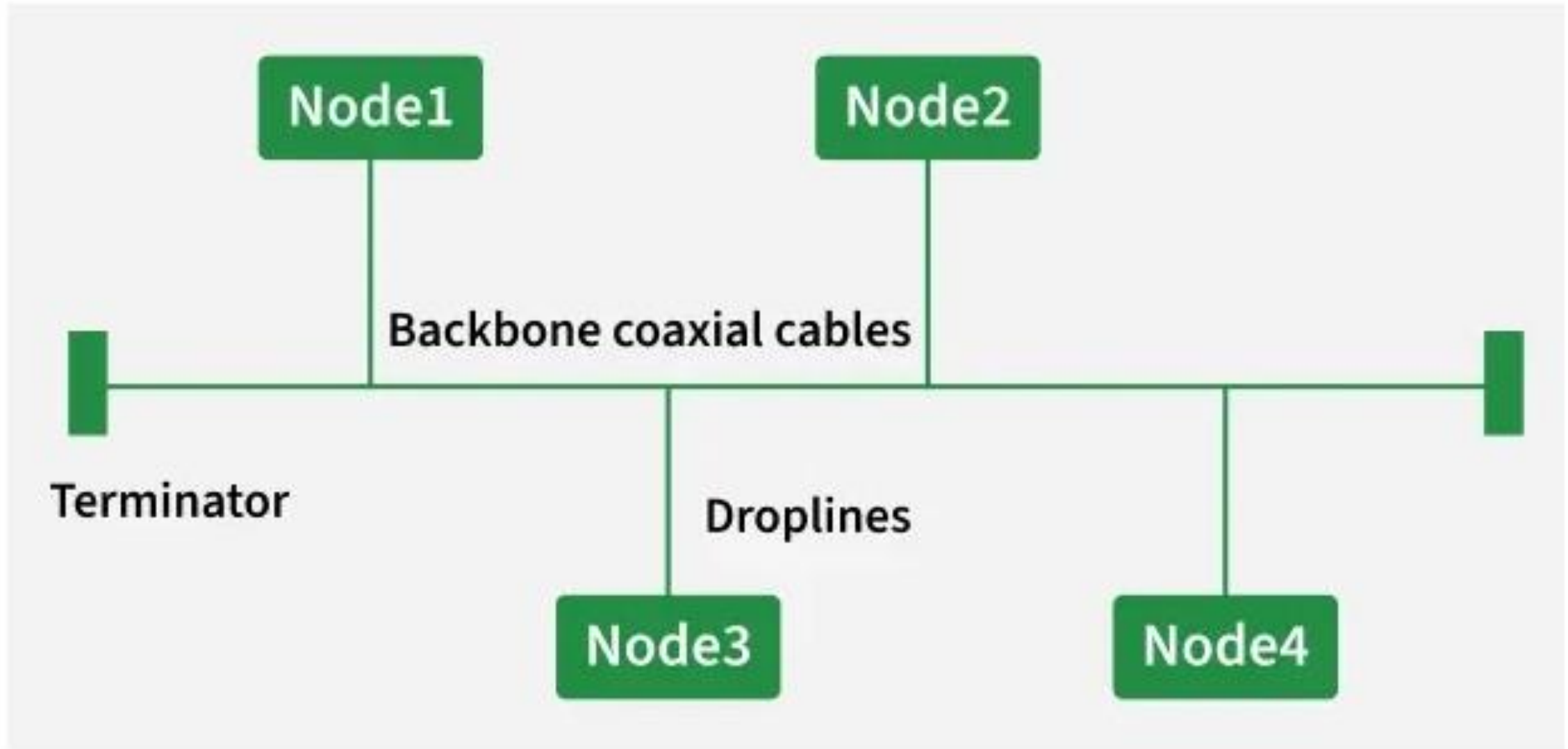
Network Topology

A network topology is the arrangement of devices (nodes) and connections (links) in a computer network. It shows how computers, servers, and other devices are connected and how data flows between them. There are two main types of topology:

- **Physical Topology:** The actual physical layout of cables and devices.
- **Logical Topology:** How data moves across the network, regardless of physical layout.

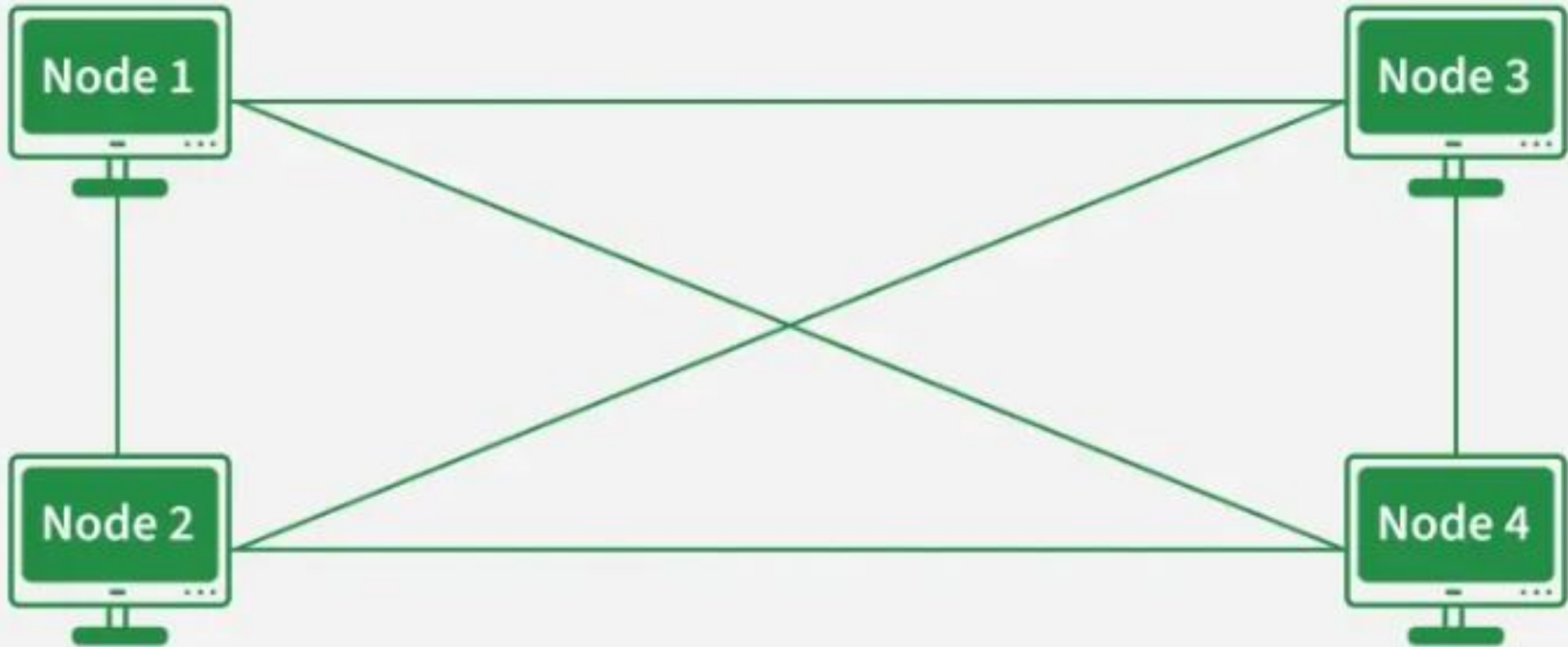
Point to Point



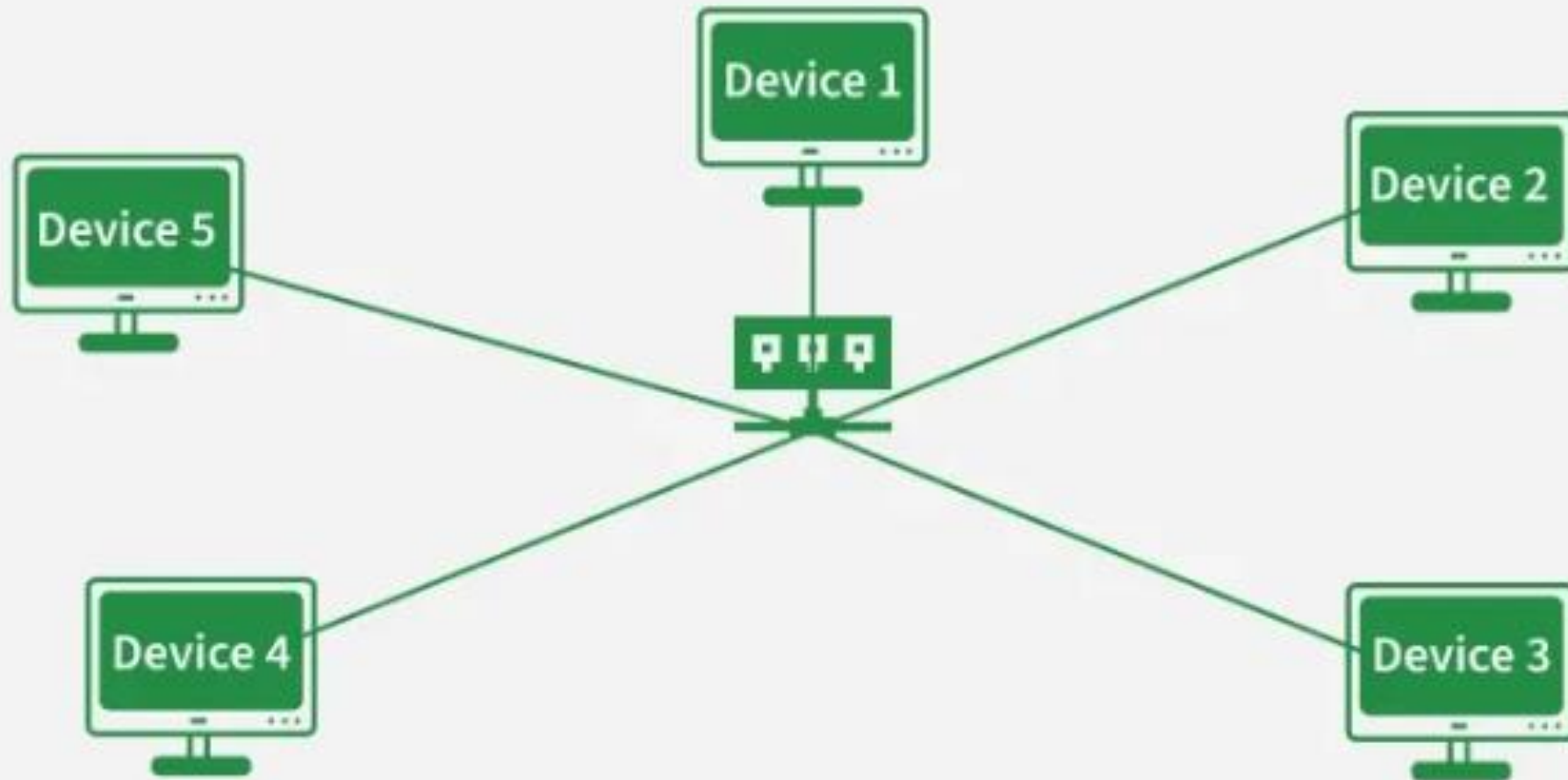


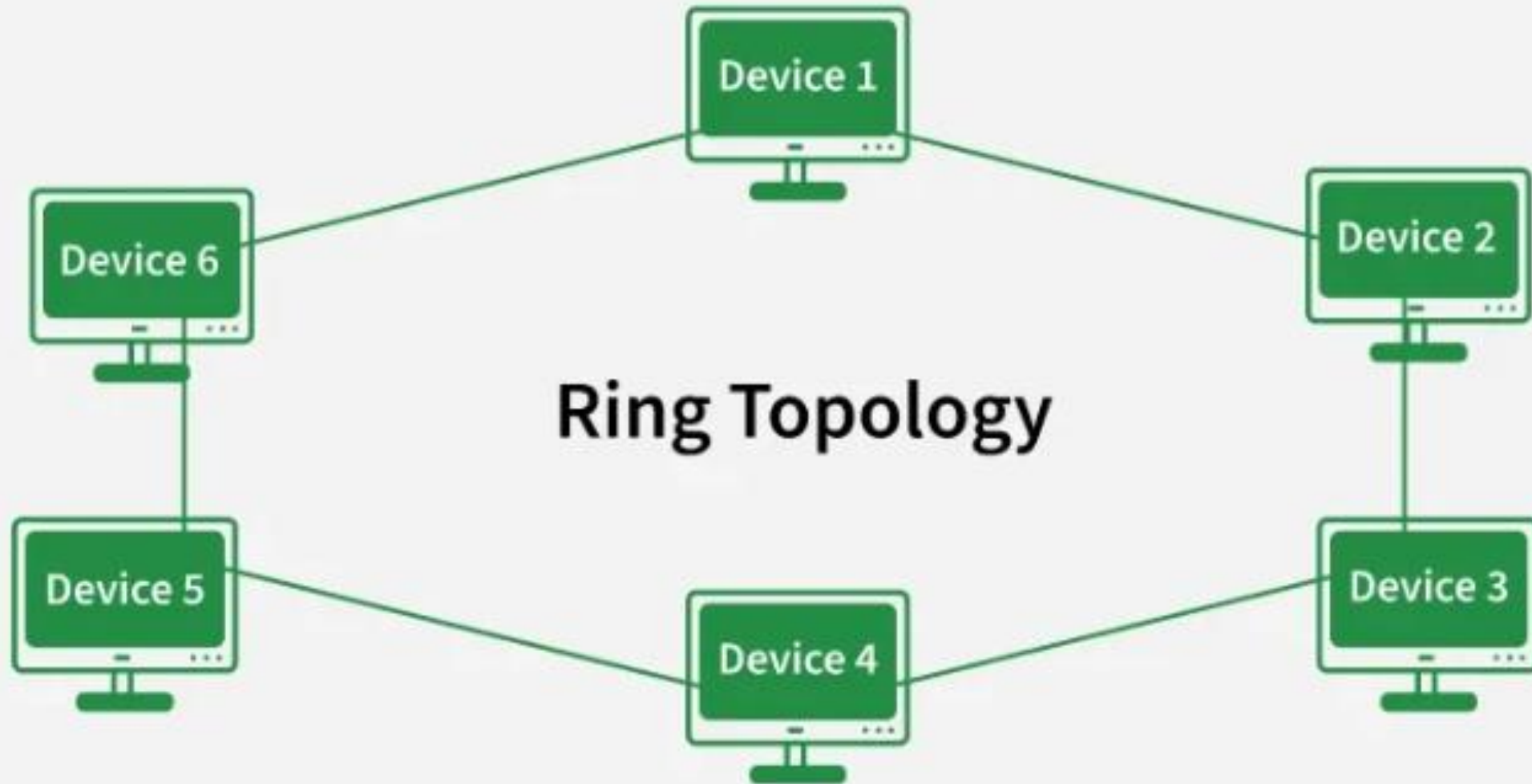
Bus

Mesh Topology

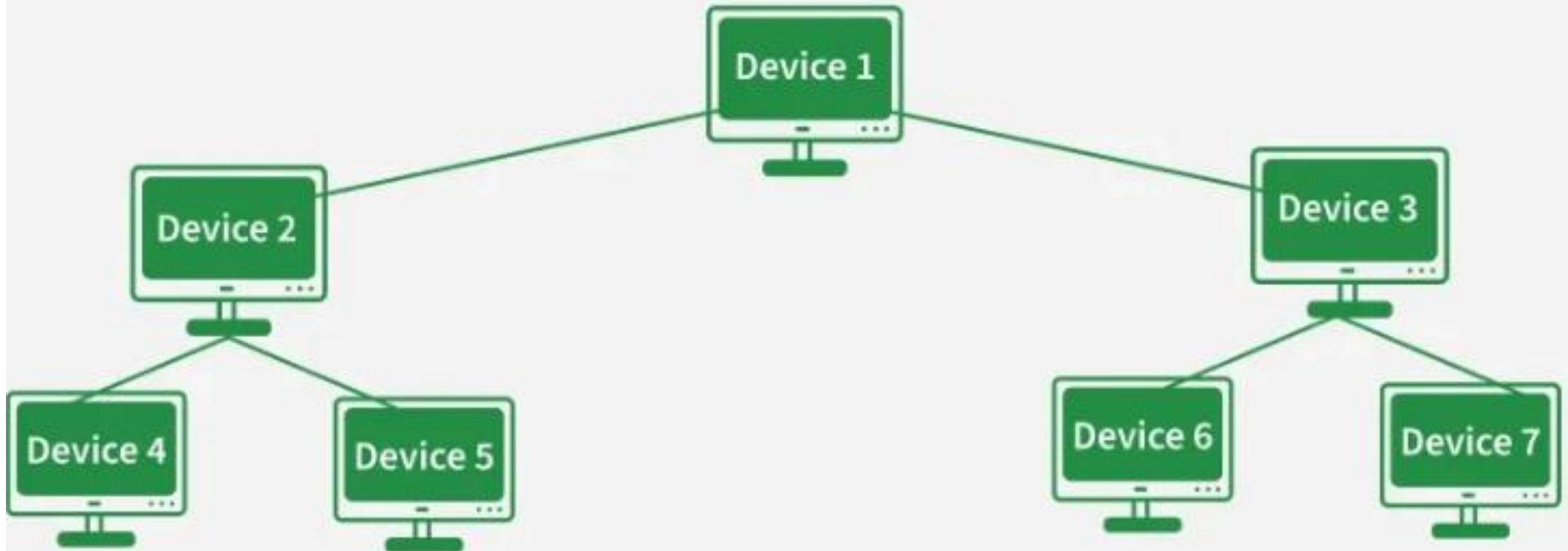


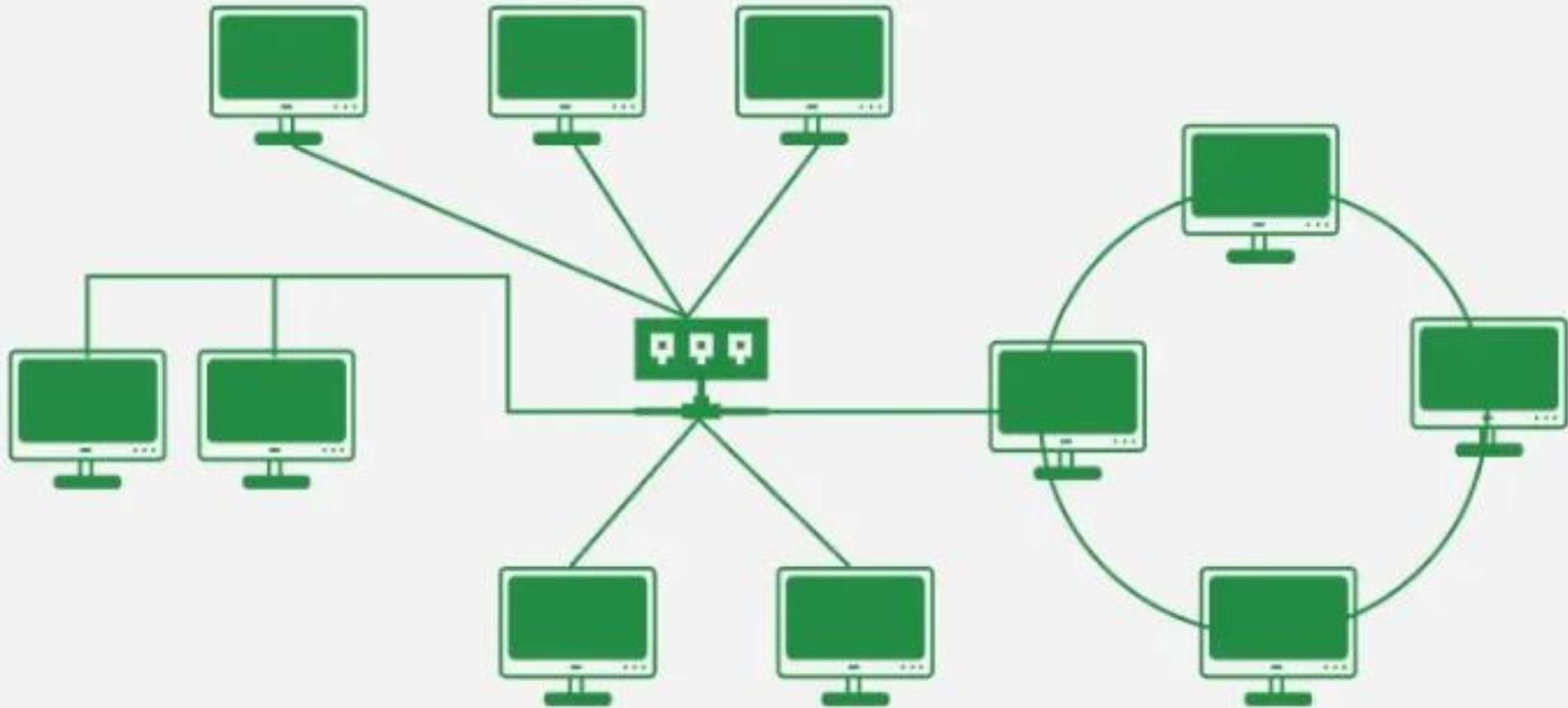
Star Topology





Tree Topology





Hybrid

Network Devices

- Router
- Switch
- Hub
- Firewall
- Gateway
- Access Points
- Modem
- Bridge
- Repeater

**Correct Order from Layer 1 → Layer 7 (Easy
Memory Trick)**

**Hub → Repeater → Modem → Bridge → Switch →
Access Point → Router → Gateway → Firewall
/ Proxy**

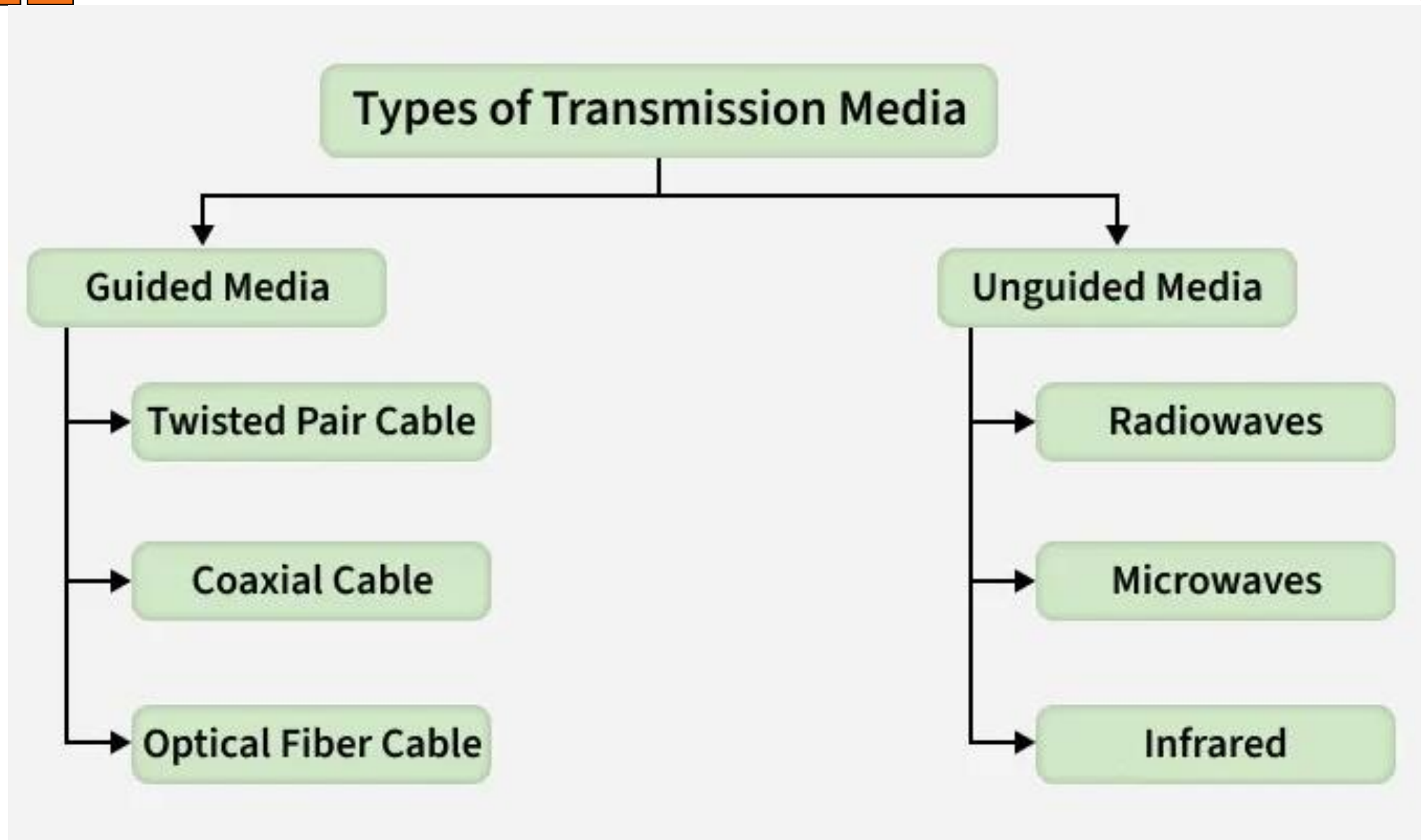
OSI Layer	Device
Layer 1 - Physical	Hub, Repeater
Layer 2 - Data Link	Bridge, Switch, Access Point
Layer 3 - Network	Router
Layer 4 - Transport	Firewall (sometimes operates here)
Layer 5-6 - Session/Presentation	Gateway
Layer 7 - Application	Proxy
Special (works across layers)	Modem

Transmission Media in Computer Networks

Transmission media refers to the physical or wireless communication channel used to carry data signals from one device to another within a computer network. It forms the fundamental pathway through which information is transmitted, ensuring connectivity between networked devices.

The selection of a transmission medium depends on factors such as:

- Transmission distance
- Data transfer speed (bandwidth)
- Susceptibility to interference and noise
- Cost and installation requirements



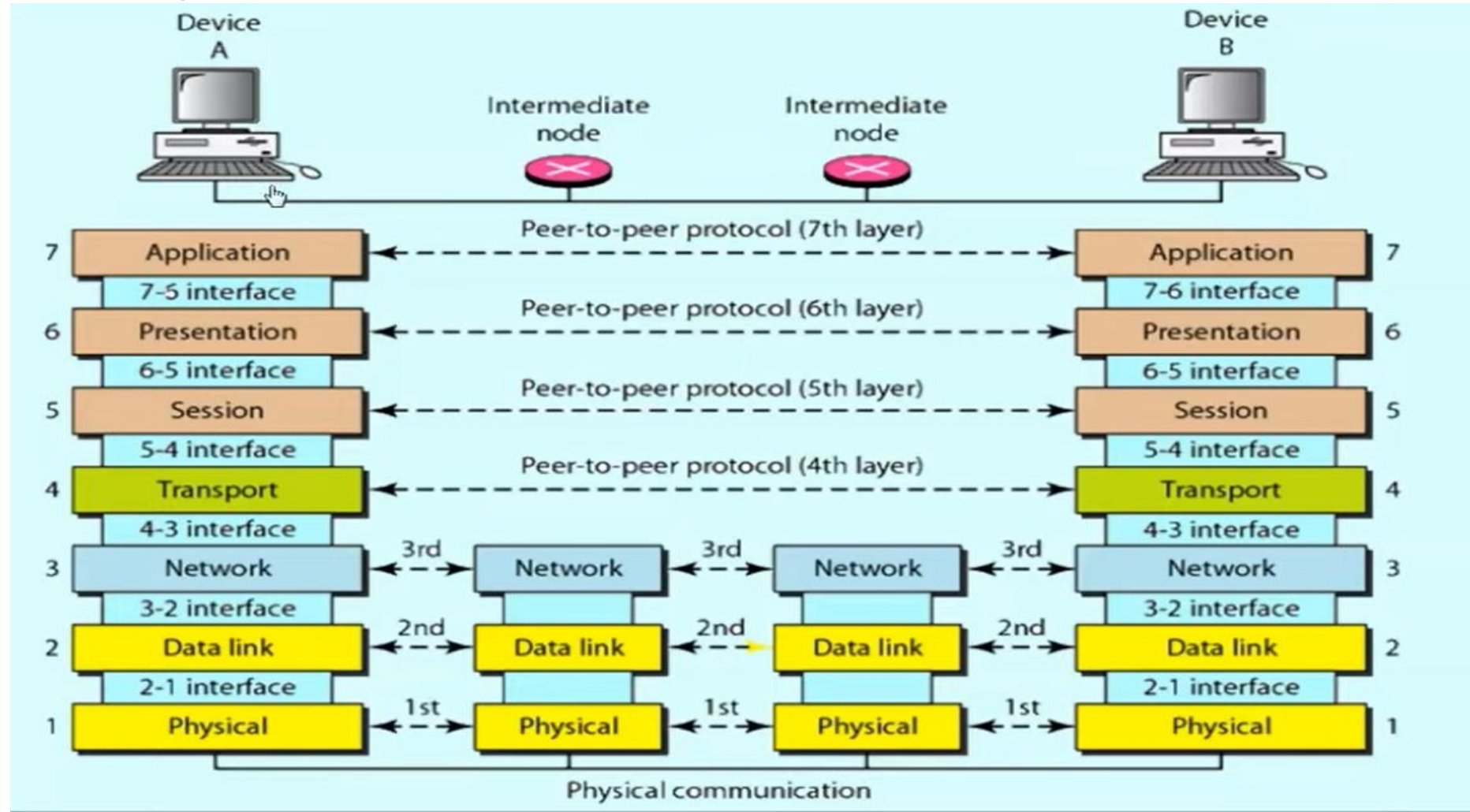
Types of Transmission Media

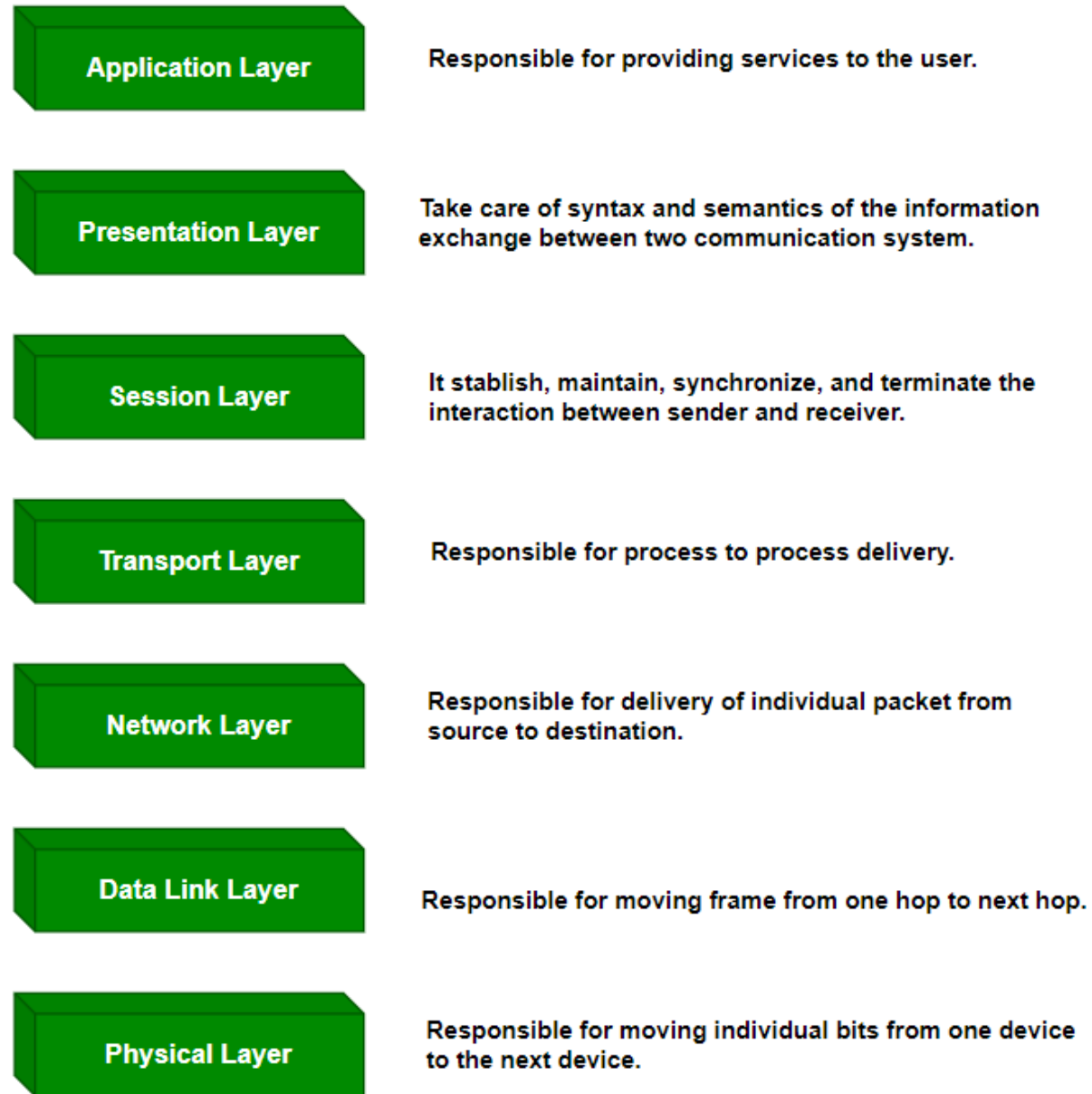
OSI Model

- OSI stands for Open Systems Interconnection.
- It is a conceptual framework developed by the International Organization for Standardization (ISO) that defines how computers communicate over a network.
- It consists of seven layers, each with specific functions and protocols.
- The purpose of the OSI model is to show how communication between different systems.
- It can be facilitated without modifying the logic of the underlying hardware and software

OSI Reference Model Layers

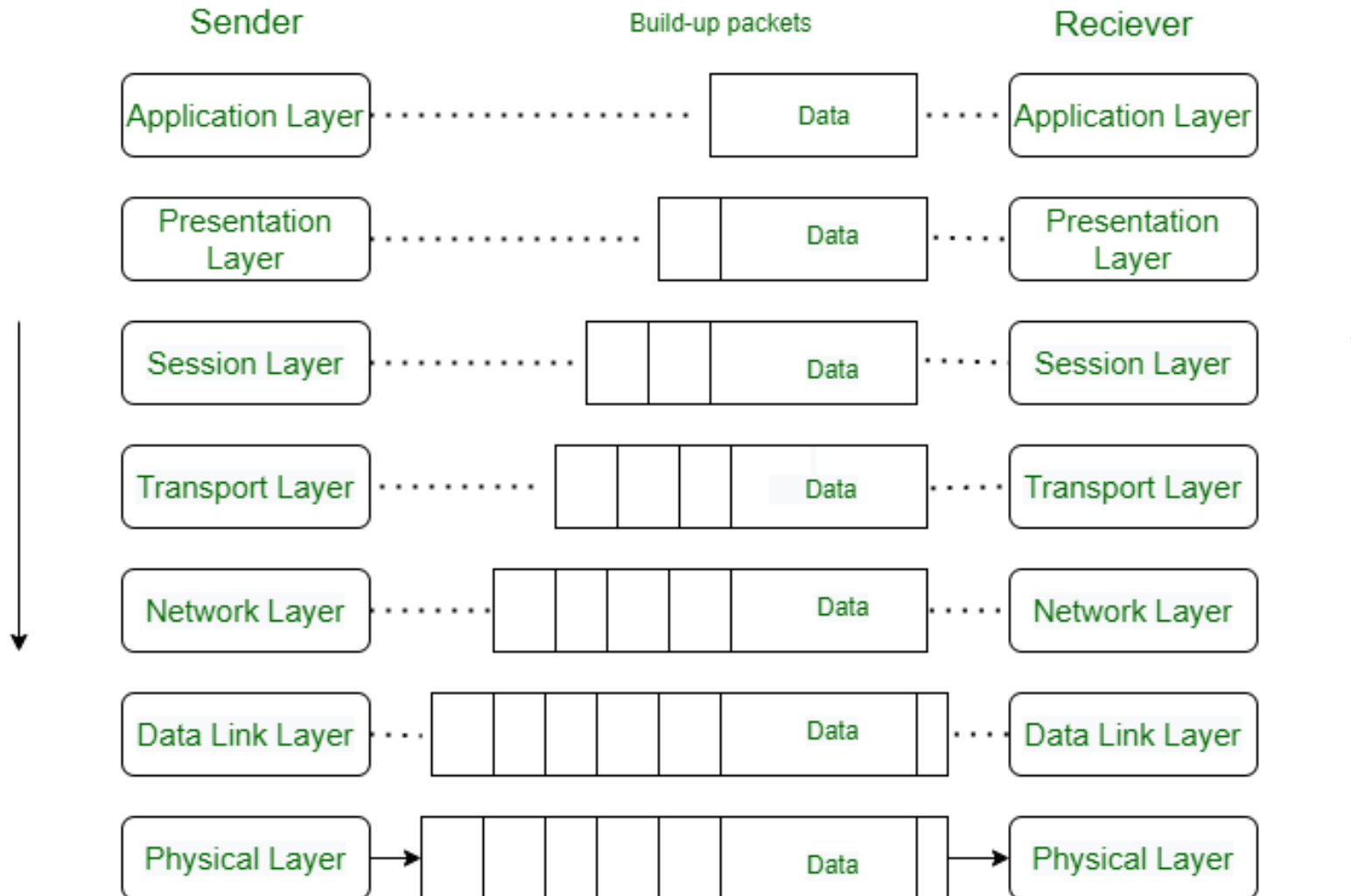
- Application Layer
- Presentation Layer
- Session Layer
- Transport Layer
- Network Layer
- Data Link Layer
- Physical Layer





- **Physical Layer:** Sends raw data (like bits, which are 1s and 0s) through cables or wireless signals.
- **Data Link Layer:** Packages the raw data into chunks called frames and checks for any errors.
- **Network Layer:** Finds the best route for the data to travel and adds addresses (like [IP addresses](#)) so it knows where to go.
- **Transport Layer:** Breaks the data into smaller pieces (called segments) and ensures everything arrives correctly and in order.
- **Session Layer:** Manages the conversation between two devices, making sure they stay connected during the data exchange.
- **Presentation Layer:** Translates the data into a format that the receiving device can understand, like converting text or images.
- **Application Layer:** This is where the user interacts with the data through apps like web browsers, email, or games.

How Data Flows Through the OSI Model



What is Bit Stuffing?

Bit stuffing is a technique used in the **Data Link Layer** to **avoid confusion between actual data and the flag sequence**.

Flag Pattern :

01111110

If this pattern **appears inside data**, the receiver may think the frame has ended.

☞ So, we **insert a 0 after every 5 consecutive 1s** in the data.

Why Bit Stuffing is Needed?

Suppose **actual data contains:**

01111110

This looks **exactly like the flag** → **✗** receiver gets confused.

So, **bit stuffing avoids this ambiguity.**

Example 1 (Simple)

Original Data:

1101111110110

Step-by-step Bit Stuffing:

Look for **five continuous 1s:**

110 11111 1 0110

Now insert **0** after five 1s:

110 11111 0 1 0110

Final Stuffed Data:

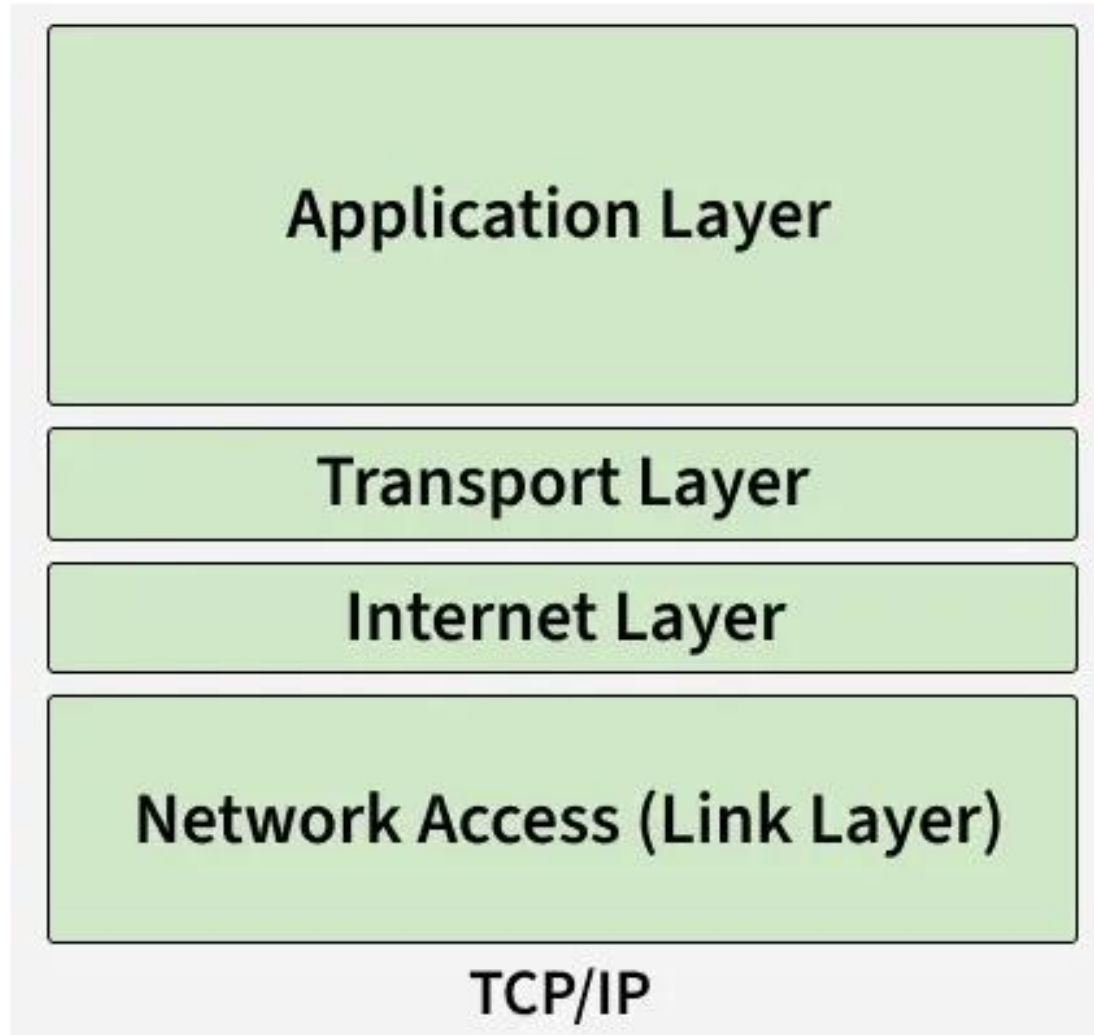
11011111010110

TCP/IP Model

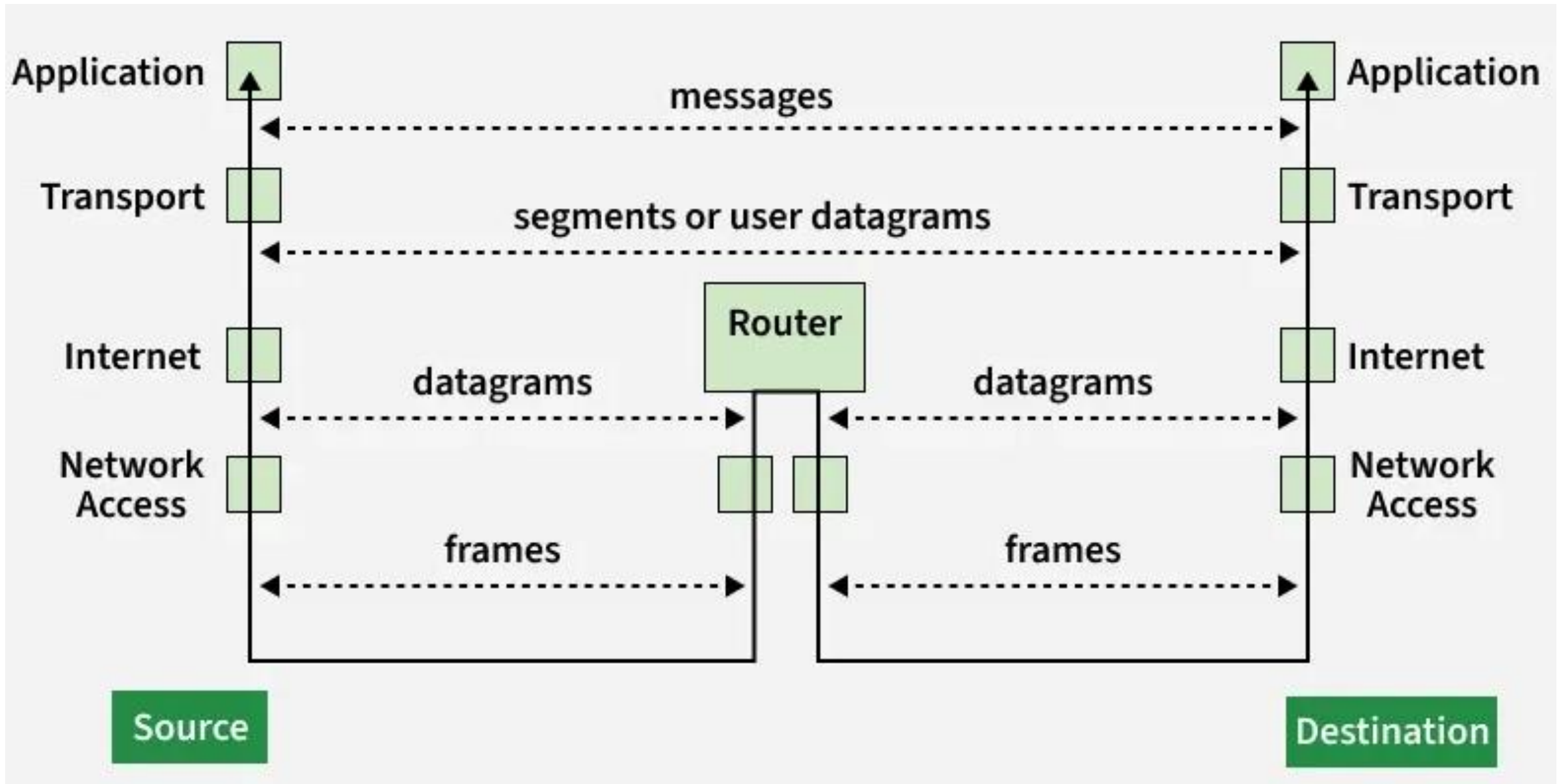
The TCP/IP model is a framework that is used to model the communication in a network. It is mainly a collection of network protocols and organization of these protocols in different layers for modeling the network.

- The original TCP/IP standard (defined in RFC 1122) consists of four layers: Application, Transport, Internet, and Network Access.
- In practice and teaching, it is often shown as a 5-layer model by separating the Physical layer from the Network Access layer. This helps clearly distinguish physical hardware (cables, signals) from data framing and MAC addressing.
- While the [OSI model](#) has seven layers, the 5 layer TCP/IP model is simpler and commonly used in today's Internet and networking systems.

Layers of TCP/IP Model



TCP/IP Layers



When Sending Data (From Sender to Receiver)

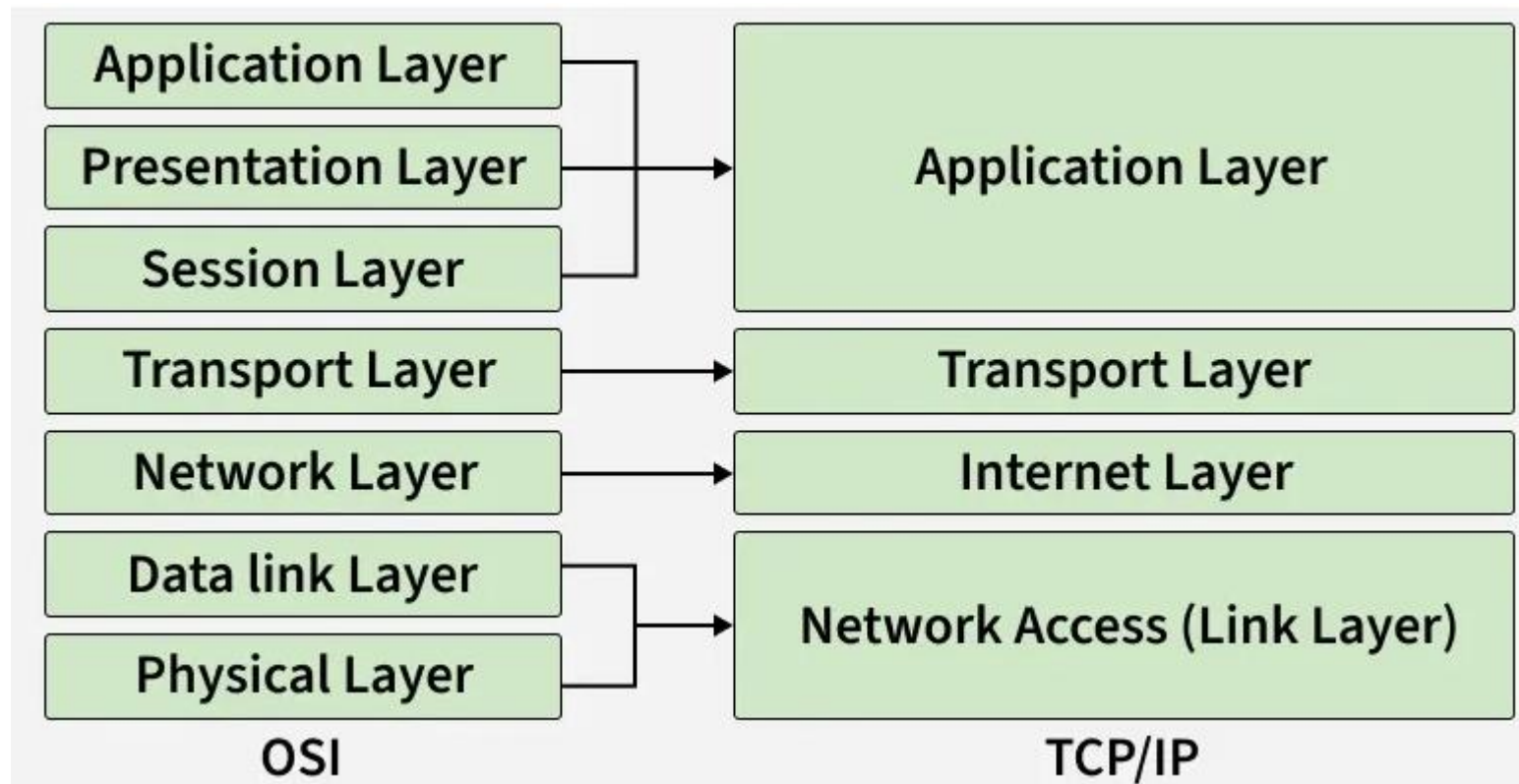
- **Application Layer:** Prepares user data using protocols like HTTP, FTP, or SMTP.
- **Transport Layer (TCP/UDP):** Breaks data into segments and ensures reliable (TCP) or fast (UDP) delivery.
- **Internet Layer (IP):** Adds IP addresses and decides the best route for each packet.
- **Link Layer (Network Access Layer):** Converts packets into frames and sends them over the physical network.

When Receiving Data (At the Destination)

- **Link Layer:** Receives bits from the network and rebuilds frames to pass to the next layer.
- **Internet Layer:** Checks the IP address, removes the IP header, and forwards data to the Transport Layer.
- **Transport Layer:** Reassembles segments, checks for errors, and ensures data is complete.
- **Application Layer:** Delivers the final data to the correct application (e.g., displays a web page in the browser).

Why TCP/IP is Used Over the OSI Model

TCP/IP is used over the OSI model because it is simpler, practical, and widely adopted for real-world networking and the internet. The diagram below shows the comparison of OSI layer with the TCP:



Subnetting (Computer Networks)

Subnetting is the process of dividing a large IP network into smaller, manageable sub-networks called **subnets**. It helps in efficient IP address utilization, improves network performance, and enhances security.

Why Subnetting is Needed

1. **Efficient use of IP addresses** – avoids wastage of IPs
2. **Reduced network traffic** – smaller broadcast domains
3. **Better security** – isolation between departments/networks
4. **Simplified network management**

TCP/IP Protocol Suite

- IP Addressing (IPv4 and IPv6)
- IPv4 Addressing
- IPv4 Components
 - Network Portion
 - Host Portion
- IPv4 Classes

Class	Fixed Bits	NID Bits	HID Bits	Network ID Range	Host ID Range
A	0	8	24	1.0.0.0 - 126.255.255.255	1.0.0.1 - 126.255.255.254
B	10	16	16	128.0.0.0 - 191.255.255.255	128.0.0.1 - 191.255.255.254
C	110	24	8	192.0.0.0 - 223.255.255.255	192.0.0.1 - 223.255.255.254
D	1110	N/A	N/A	Reserved for Multicast	N/A
E	1111	N/A	N/A	Reserved for Experimental	N/A

IP Address

An **IPv4 address** is 32 bits long and written in dotted decimal format:

192.168.1.10

It has two parts:

- **Network ID**
- **Host ID**

Class	First octet value	Subnet mask
A	0-127	8
B	128-191	16
C	192-223	24
D	224-239	-
E	240-255	-

Class	1st octet of IP address	Default Subnet Mask	Network / Host	Number of networks	Maximum nodes in a network
A	1 - 126	255.0.0.0	N.H.H.H	126	16,777,214
B	128 - 191	255.255.0.0	N.N.H.H	16,384	65,534
C	192 - 223	255.255.255.0	N.N.N.H	2,097,152	254
D	224 - 239				
E	240 - 254				

Formula Used

- Hosts per subnet = $2^h - 2$
- Subnet prefix = $32 - h$
- Number of subnets = 2^n

Numerical 1: Number of Hosts from CIDR

Statement

How many usable hosts are available in a /**28** network?

$$\text{Hosts} = 2^h - 2$$

$$h = 32 - 28 = 4$$

$$\text{Hosts} = 2^4 - 2 = 14$$

Numerical 2: Subnet Creation (Subnet-based)

Statement

Given IP address **192.168.10.0/24**.

Create **8 subnets**. Find:

1. New subnet mask
2. Number of hosts per subnet
3. Network & broadcast address of each subnet

Formula Used

- Number of subnets = 2^n
- Hosts per subnet = $2^h - 2$
- New prefix length = Original prefix + n



Default subnet mask = 255.255.255.0

CIDR notation = /24

This means:

- **24 bits → Network portion**
- **8 bits → Host portion**

Binary representation of last octet:

00000000 (8 host bits)

Create **8 subnets**.

Formula:

Number of subnets = 2^n

Try values of n :

$$2^3 = 8 \quad \checkmark$$

✓ Borrow 3 bits from host portion

Why we borrow bits

Borrowing bits from the host portion increases the number of subnets.

Original CIDR = /24

Borrowed bits = 3

New CIDR = 24 + 3 = /27

Convert /27 to dotted decimal:

Binary of last octet:

11100000

Decimal:

128 + 64 + 32 = 224

✓ New subnet mask = 255.255.255.224

Hosts per subnet = $2^h - 2$

Where:

h = remaining host bits

h = 8 - 3 = 5

Hosts = $2^5 - 2 = 32 - 2 = 30$

✓ Each subnet has 30 usable hosts



Subnet	Network Address	Usable IP Range	Broadcast Address
1	192.168.10.0	.1 – .30	.31
2	192.168.10.32	.33 – .62	.63
3	192.168.10.64	.65 – .94	.95
4	192.168.10.96	.97 – .126	.127
5	192.168.10.128	.129 – .158	.159
6	192.168.10.160	.161 – .190	.191
7	192.168.10.192	.193 – .222	.223
8	192.168.10.224	.225 – .254	.255

TAKEAWAYS

- ✓ Class C default mask = /24
- ✓ Borrow 3 bits → 8 subnets
- ✓ New mask = /27 (255.255.255.224)
- ✓ Block size = 32
- ✓ Hosts per subnet = 30

Supernetting(Route Aggregation)

- is a method used in computer networking to combine multiple smaller subnets into a single larger network.
- Why Supernetting?
 - Reduced Routing Table Size
 - Simplified Network Configuration
- Address Space Conservation

Go-Back-N (GBN) Protocol

Definition:

Go-Back-N (GBN) is a **sliding window based data link layer protocol** used for **reliable data transmission**, where the sender can send **multiple frames before receiving acknowledgments**, but the receiver accepts **only in-order frames**.

If an error occurs, the sender **retransmits the erroneous frame and all frames sent after it**.

Working Principle (Step-by-Step)

1. Sender transmits **multiple frames continuously** up to window size **N**.
2. Receiver:
 1. Accepts **only the next expected frame**.
 2. Discards **out-of-order frames**.
3. If a frame is **lost or damaged**:
 1. Receiver sends **ACK for last correctly received frame**.
4. Sender:
 1. **Goes back** to the lost frame.
 2. Retransmits **that frame and all following frames**.

Sender Side

- The sender maintains a sliding window of size N (e.g., in GBN-4, the window size is 4).
- It can transmit up to N frames without waiting for individual ACKs.
- Each transmitted frame is associated with a timer.
- When an ACK is received, the sender slides the window forward, allowing transmission of new frames.
- If a timeout occurs for a frame, the sender retransmits that frame and all subsequent frames in the current window.

Receiver Side

- The receiver maintains a window size of 1 ($WR = 1$).
- It accepts only the next expected frame in sequence.
- If the expected frame is received correctly, then the receiver sends an ACK, and updates the expected sequence number.
- If an out-of-order frame is received, then the receiver discards the frame, and resends the ACK for the last correctly received frame.

Simple Example

Assume:

- Window size $N = 4$
- Frames: 1, 2, 3, 4, 5, 6

Transmission:

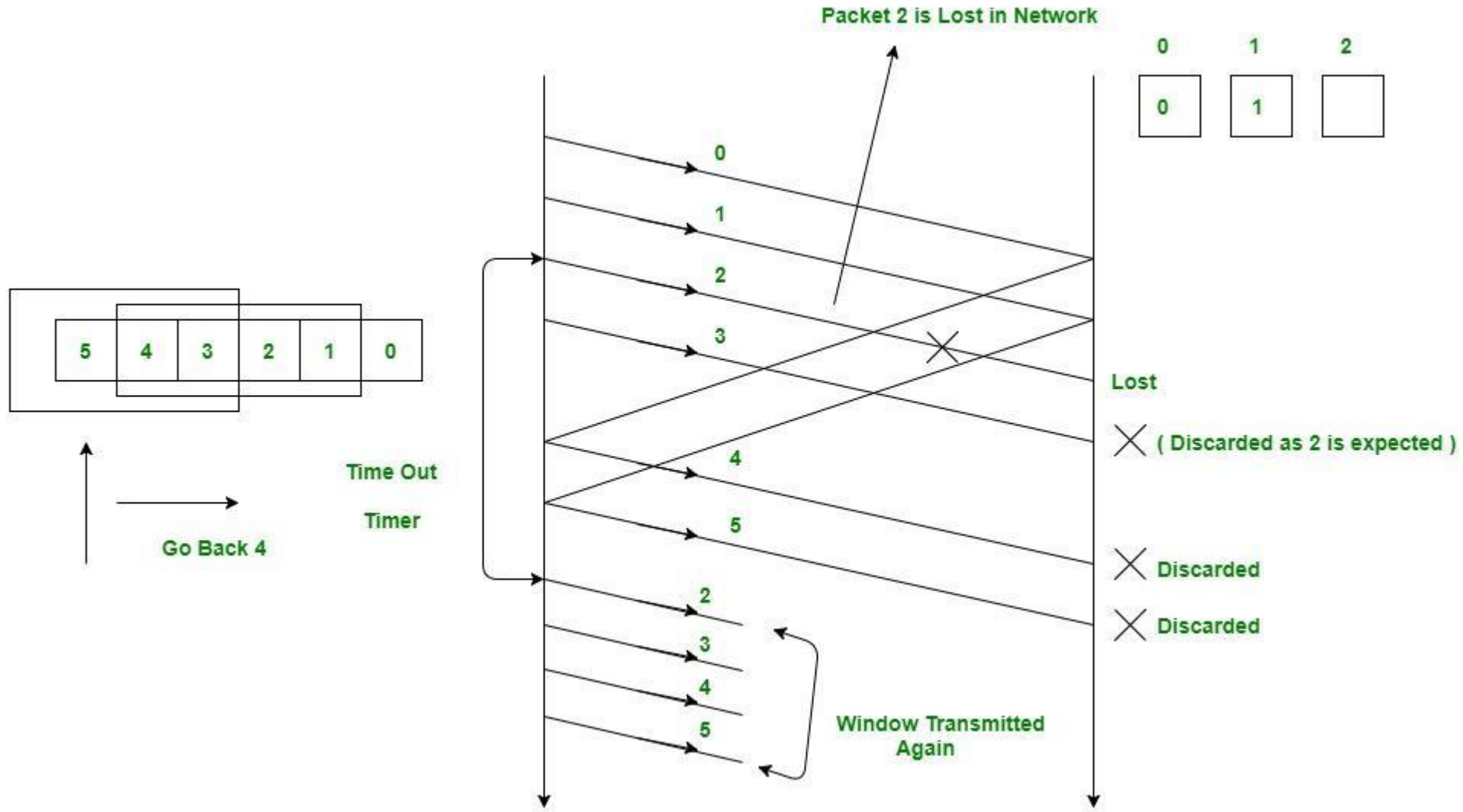
Sender sends:

👉 1 → 2 → 3 → 4

If Frame 3 is lost, then:

- Receiver gets: 1, 2, (3 lost), 4 discarded
- Receiver sends: ACK 2
- Sender retransmits: 3, 4, 5, 6

👉 This is called **Go-Back-N** → go back to frame 3.



1. Value of a (Delay Ratio)

$$a = \frac{\text{Propagation Delay}}{\text{Transmission Time per Frame}}$$

Where:

- Propagation Delay = T_p
- Transmission Time = T_t

2. Transmission Time per Frame

$$T_t = \frac{\text{Frame Size (bits)}}{\text{Bandwidth (bps)}}$$

3. Channel Utilization (Efficiency) for GBN

$$U = \frac{W}{1 + 2a}$$

Where:

- U = Utilization (Efficiency)
- W = Window size
- a = Delay ratio

4. Maximum Channel Utilization Condition

$$W \geq 1 + 2a \Rightarrow U = 1 (100\%)$$

Problem:

A sender transmits data using **Go-Back-N protocol** over a channel with:

- **Bandwidth = 1 Mbps (B)**
- **Frame size = 1000 bytes (L)**
- **One-way propagation delay = 25 ms**
- **Window size = 5 frames**

Assume **ACK time negligible** and **no errors**.

Find:

(a) Value of a

(b) Channel utilization U

(a) Calculate value of a

$$a = \frac{T_p}{T_t}$$

Transmission time per frame:

$$T_t = \frac{L}{B} = \frac{8000}{1,000,000} = 0.008 \text{ s} = 8 \text{ ms}$$

Value of a :

$$a = \frac{25}{8} = 3.125$$

(b) Channel Utilization U

$$U = \frac{W}{1 + 2a}$$

$$U = \frac{5}{1 + 2(3.125)} = \frac{5}{1 + 6.25} = \frac{5}{7.25} = 0.689$$

Error Detection :

Error Detection is a technique used in computer networks to **identify whether data has been corrupted during transmission** from sender to receiver due to noise, interference, or signal distortion.

Parity Bit

A **parity bit** is an extra bit added to a group of **data bits** to help detect **errors** during data transmission. It tells whether the total number of 1s in the data is **even** or **odd**.

There are two types of parity:

1.Even Parity – The total number of 1s (including the parity bit) should be even.

2.Odd Parity – The total number of 1s (including the parity bit) should be odd.

Data: 1011

1. Even Parity:

- Count the number of 1s $\rightarrow 1 + 0 + 1 + 1 = 3$ (odd)
- To make it even, parity bit = 1
- **Data sent: 10111**

2. Odd Parity:

- Count the number of 1s $\rightarrow 3$ (odd)
- To make it odd, parity bit = 0
- **Data sent: 10110**

Cyclic Redundancy Check (CRC)

Cyclic Redundancy Check (CRC) is a method of **error detection** used in digital networks and storage devices.

- It detects **accidental changes/errors** in transmitted data.
- It treats the data as a **binary number** and divides it by a **pre-agreed binary number** called the **generator polynomial**.
- The **remainder** of this division is called the **CRC** or **checksum**.
- The CRC is sent along with the data. The receiver performs the same division. If the remainder is **0**, the data is **error-free**.

How CRC Works (Simplified Steps)

1. Suppose data to transmit = 110101
2. Choose a generator (agreed polynomial) = 1011
3. Append zeros to the data (number of zeros = length of generator - 1)
 - Data becomes 110101000 (3 zeros for 4-bit generator)
4. Divide data by generator using binary division (XOR)
 - Ignore carries in XOR
 - The remainder after division is the CRC
5. Send the data + CRC
 - Suppose remainder = 011 → transmitted data = 110101011
6. Receiver divides received data by same generator
 - If remainder = 0 → no error
 - If remainder $\neq$ 0 → error detected



Given

- Data: 1101
- Generator: 101 (3 bits)

Step 1: Append zeros

Generator length = 3, so append $(3-1) = 2$ zeros.

1101 → 110100

Step 2: Divide 110100 by 101 (XOR division)

$$110 \oplus 101 = 011$$

Bring down next bit → 111

$$111 \oplus 101 = 010$$

Bring down next bit → 100

$$100 \oplus 101 = 001$$

Final remainder = 10

Step 3: Append remainder to original data

```
Data = 1101
CRC  = 10
-----
Transmitted frame = 110110
```

So the correct transmitted data should be:

110110

NOT 110111 as shown in the image.

Step 4: Receiver check

Receiver divides $110110 \div 101$

Remainder:

00

Since remainder = 0, the data is correct.

Binary	Polynomial
1101 (data)	$x^3 + x^2 + 1$
1011 (gen)	$x^3 + x + 1$
1101000	$x^6 + x^5 + x^3$
CRC = 001	1
Transmit	1101001

Checksum

A **Checksum** is a simple method of **error detection** used to ensure data integrity during transmission.

- It **sums up the data bits/bytes** in a message and sends the sum (or a modified version) along with the data.
- The receiver performs the same sum and compares it with the transmitted checksum.
- If they match → data is correct; if not → **error detected**.

Basic Principle

At Sender:

1. Divide data into blocks
2. Add all blocks
3. Take **1's complement**
4. Send **Data + Checksum**

At Receiver:

1. Add all received blocks + checksum
2. If result = **all 1s** → **No Error**
3. Else → **Error detected**

Data blocks: 1001, 0101, 0011

Sum: $1001 + 0101 + 0011 = 0010$

Checksum: 1101 (1's complement)

Block 1 = 1001

Block 2 = 0101

Block 3 = 0011

1101 (Checksum)

Hamming Code

Hamming Code is an **error-detecting and error-correcting code**.

- It can **detect and correct single-bit errors** in data transmission.
- It adds **redundant (parity) bits** to the data in **specific positions**.

In **Hamming code**, the **parity bits** are systematically placed at positions that are **powers of 2** within the encoded bit sequence.

Technique	Number of Bits Detected / Corrected	Advantages	Disadvantages
Parity Bit	Detects single-bit errors (cannot detect if 2 bits flipped)	- Very simple to implement- Low overhead (1 extra bit)	- Cannot correct errors- Cannot reliably detect multiple-bit errors
Hamming Code	Detects single-bit errors and corrects single-bit errors; can detect 2-bit errors but cannot correct them	- Can correct single-bit errors- Efficient for small data blocks	- More overhead than parity (needs multiple parity bits)- Not suitable for large errors or burst errors
Cyclic Redundancy Check (CRC)	Detects single-bit, double-bit, and burst errors (depending on generator polynomial)	- Very reliable- Detects burst errors efficiently- Widely used in networking	- Slightly more complex to implement- Cannot correct errors, only detects them
Checksum	Detects some single-bit errors and some multiple-bit errors, but not reliable for burst errors	- Simple and fast- Easy to implement in software	- Less reliable than CRC- Cannot detect all errors- Cannot correct errors

Network Performance Metrics – Delays

In computer networks, **delay (latency)** is the time taken by data to travel from **source to destination**.

Main types of delay:

1. Transmission Delay
2. Propagation Delay
3. Queuing Delay
4. Processing Delay

☞ **Total End-to-End Delay = Sum of all above delays**

(a) Transmission Delay

Definition:

Time required to push all bits of a data packet onto the transmission medium (link).

Formula:

$$\text{Transmission Delay} = \frac{\text{Packet Size (bits)}}{\text{Bandwidth (bps)}}$$

Numerical Example 1:

Packet size = 1000 bytes

Bandwidth = 1 Mbps

Convert bytes → bits:

$$1000 \times 8 = 8000 \text{ bits}$$

$$\text{Transmission Delay} = \frac{8000}{1 \times 10^6} = 0.008 \text{ sec} = 8 \text{ ms}$$

(b) Propagation Delay

Definition:

Time taken by signal to travel from sender to receiver through the physical medium.

Formula:

$$\text{Propagation Delay} = \frac{\text{Distance}}{\text{Speed of signal}}$$

Distance-Based Numerical Example 1:

Distance = 2000 km = 2×10^6 m

Speed = 2×10^8 m/s

$$\text{Delay} = \frac{2 \times 10^6}{2 \times 10^8} = 0.01 \text{ sec} = 10 \text{ ms}$$

✓ Answer: 10 ms

Numerical Example 2:

Distance = 500 km

Speed = 3×10^8 m/s

$$= \frac{5 \times 10^5}{3 \times 10^8} = 1.67 \text{ ms}$$

(c) Queuing Delay

Definition:

Time packet waits inside router/switch buffer before transmission.

Main Cause:

👉 Network Congestion

(d) Processing Delay

Definition:

Time taken by **router/switch** to:

- Check packet header
 - Perform error detection
 - Find routing path
-

Typical Range:

👉 Microseconds (μs)

Real-Life Example:

Time taken by **post office clerk** to check address and stamp.

(e) Total End-to-End Delay

Definition:

Total time taken by data to go from sender to receiver.

Formula:

Total Delay = Transmission Delay + Propagation Delay + Queuing Delay + Processing Delay

Numerical Example:

Transmission Delay = 10 ms

Propagation Delay = 20 ms

Queuing Delay = 5 ms

Processing Delay = 2 ms

$$\text{Total Delay} = 10 + 20 + 5 + 2 = 37 \text{ ms}$$

✅ Total End-to-End Delay = 37 ms

Nyquist Theorem (Noiseless Channel)

Statement:

Nyquist theorem gives the **maximum data rate** that can be transmitted over a **noiseless channel** of a given bandwidth using **finite signal levels**.

Formula:

$$C = 2B \log_2(L)$$

Symbol	Meaning
C	Maximum data rate (bps)
B	Bandwidth (Hz)
L	Number of distinct signal levels

Numerical Example

Problem:

A noiseless channel has a bandwidth of 3 kHz.

If 8 signal levels are used, find the maximum data rate.

Given:

- $B = 3 \text{ kHz} = 3000 \text{ Hz}$
- $L = 8$

Ethernet

- is a widely used networking technology that defines the rules for constructing and operating a local area network (LAN).
- Key Points about Ethernet
 - Topology
 - Data Link Layer
 - Frame Format
 - CSMA/CD

Feature	10Base-T	100Base-TX	100Base-2
Speed	10 Mbps	100 Mbps	100 Mbps
Cable	Twisted Pair	Cat-5/6 UTP	Coaxial
Distance	100 m	100 m	185 m
Topology	Star	Star	Bus
Usage	Old LAN	Modern LAN	Obsolete

Routing

- is the process of selecting the best path for network traffic to travel from the source to the destination in a computer network.
- Key Aspects of Routing
 - Routing Process
 - Source and Destination
 - Packet Forwarding
 - Routing Tables
 - Information Repository
 - Decision Making
 - Routing Algorithms
 - Dynamic Routing
 - Static Routing

There are **two main types of routing**:

a) Static Routing: In static routing, routes are manually configured by the network administrator. The router always follows the same fixed path.

Characteristics

Manual configuration

Fixed path

No automatic updates

Suitable for small networks

b) Dynamic Routing

- Routers automatically discover the **best path** using **routing protocols**.

Characteristics

- Automatic route updates
- Path changes if link fail
- suitable for large networks.

Routing Algorithms

- Distance Vector Routing
- Bellman-Ford Algorithm
- RIP (Routing Information Protocol)
- OSPF (Open Shortest Path First)
- Link-State Routing Algorithm
- Distance-Vector Routing Algorithm
- Path Vector Routing Algorithm
- BGP (Border Gateway Protocol)

1. Distance Vector Routing Algorithm (DVRA)

Definition

In Distance Vector routing, each router shares its routing table with its neighboring routers. Routers update their routing tables based on information received from neighbors.

Distance = cost / hops

Vector = direction (next hop router)

Working Principle (Step-by-step)

1. Each router maintains a routing table.
2. It sends this table to its immediate neighbors.
3. Neighbors update their tables using Bellman-Ford algorithm.
4. This process repeats until all routers get best paths.

2. Link State Routing Algorithm (LSRA)

Definition

In Link State routing, each router knows the complete network topology and calculates the shortest path using Dijkstra's algorithm.

Working Principle (Step-by-step)

1. Each router discovers its neighbors and link cost.
2. Router sends this info to all routers in the network (flooding).
3. All routers build a complete network map.
4. Best path is calculated using Dijkstra's algorithm.

IP Routing

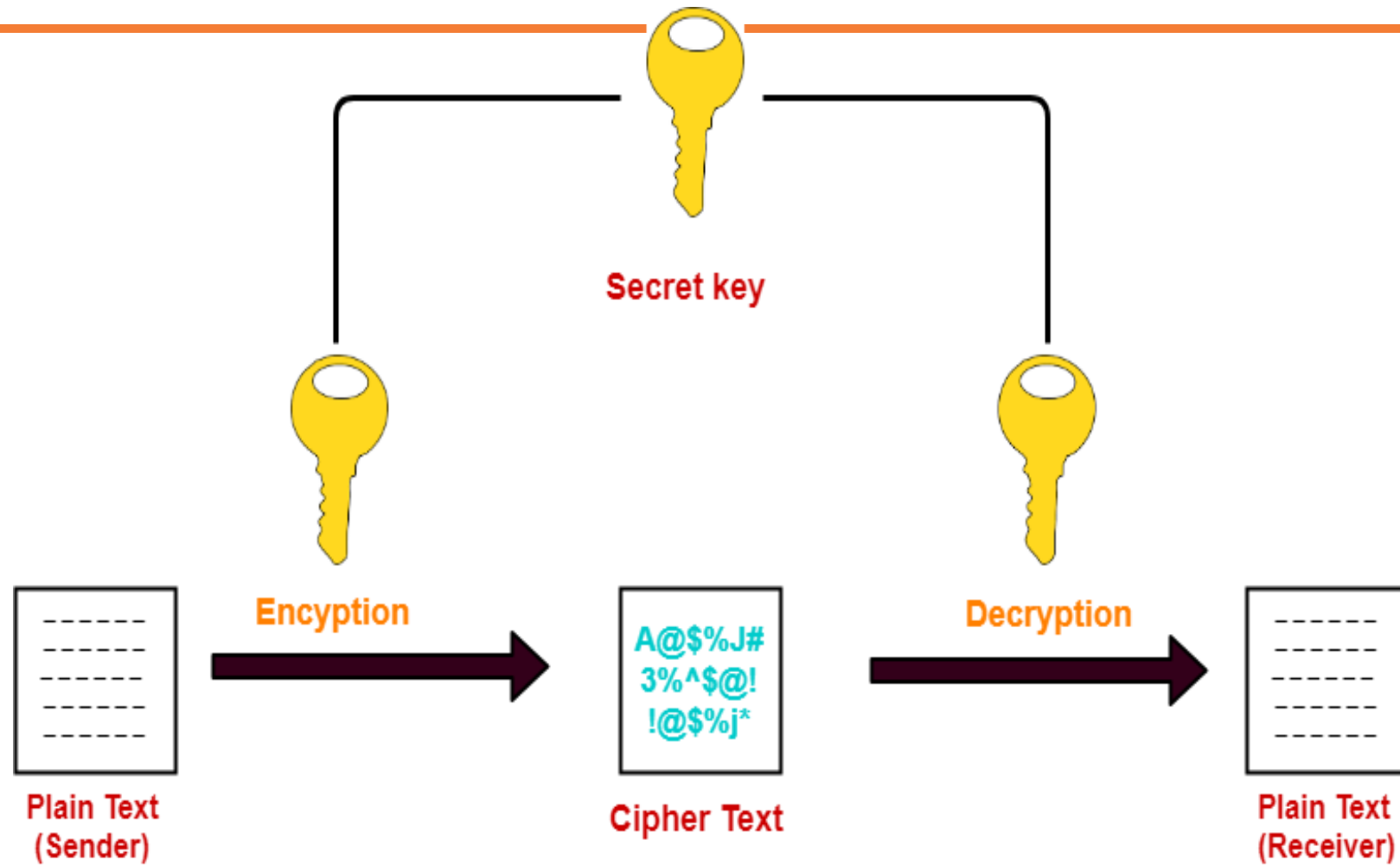
- is the process of forwarding packets from one network to another based on their destination IP addresses.
- Key Components of IP Routing
 - Routing Table
 - Routing Protocols:
 - Routing Metrics
 - Static Routing

Cryptography.



Plain Text---→ Encryption----→ Cipher Text-----→ Decryption----→ Plain Text

- Symmetric Key Cryptography
- Asymmetric Key Cryptography

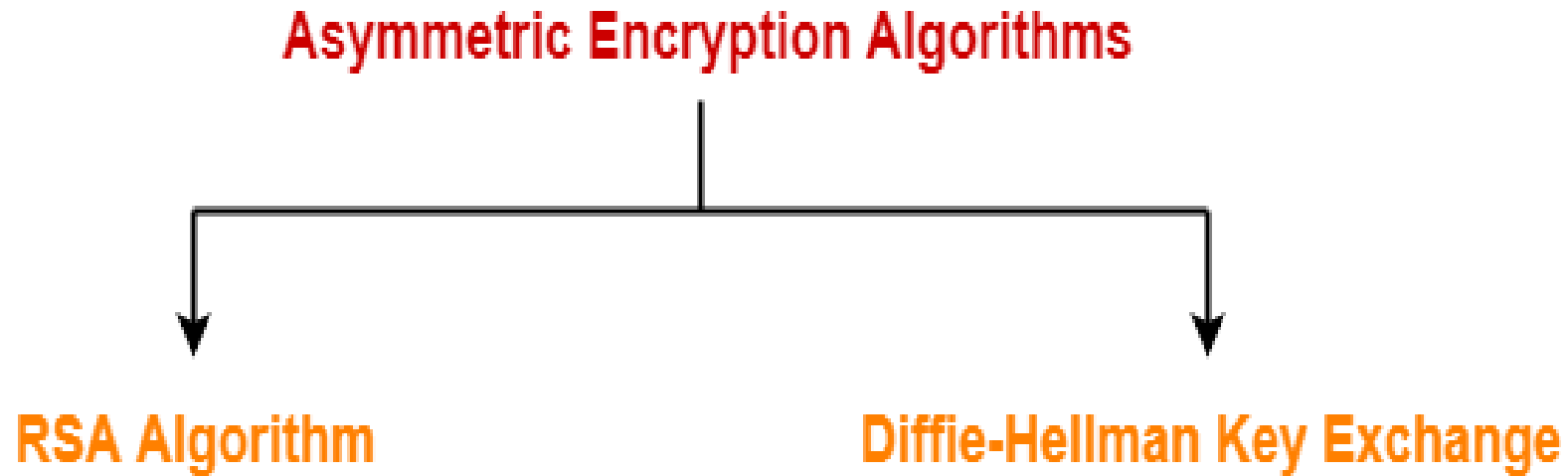


Symmetric Key Cryptography

Asymmetric Encryption-

In asymmetric encryption,

- Sender and receiver use different keys to encrypt and decrypt the message.
- The famous asymmetric encryption algorithms are-



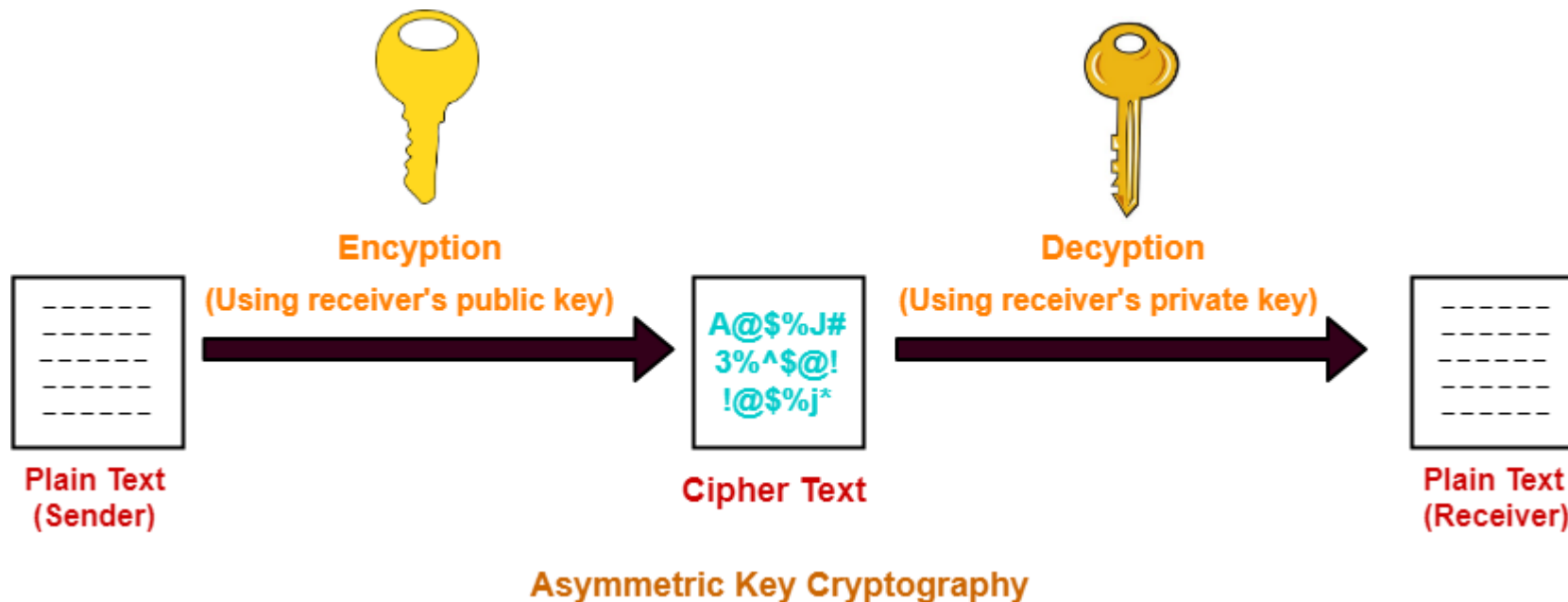
Asymmetric Key Cryptography-

In this technique,

- Sender and receiver use different keys to encrypt and decrypt the message.
- It is called so because sender and receiver use different keys.
- It is also called as **public key cryptography**.

Working-

The message exchange using public key cryptography involves the following steps-



Key Protocols: HTTP, SMTP, POP/IMAP, FTP, DNS, DHCP

1. HTTP (HyperText Transfer Protocol)

Definition

HTTP is an **application-layer protocol** used for communication between a **web browser (client)** and a **web server** to transfer web pages and resources.

Purpose

- Used to **access websites**
- Transfers **HTML pages, images, videos, CSS, JS**

How it works

1. Client sends an HTTP request (GET/POST)
2. Server sends an HTTP response

Common Methods

- **GET** – request data
- **POST** – send data
- **PUT** – update data
- **DELETE** – remove data

Port Number

- **80** – HTTP
- **443** – HTTPS (secure)

Example

Typing `http://www.google.com` in a browser.

2. SMTP (Simple Mail Transfer Protocol)

Definition

SMTP is used to **send emails** from a client to a mail server or between mail servers.

Purpose

- **Sending** emails only
- Not used for reading emails

How it works

Sender writes email

SMTP transfers it to recipient's mail server

Port Numbers

- **25** – default SMTP
- **587** – secure SMTP (TLS)

Example

Sending an email via **Gmail** → **Yahoo Mail**

3. POP & IMAP (Email Receiving Protocols)

POP (Post Office Protocol – POP3)

Definition

POP is used to **download emails** from a mail server to a local device.

Characteristics

- Emails are usually **deleted from server**
- Best for **single device access**

Port

- 110** (POP3)
- 995** (Secure POP3)

Example

Downloading emails to Outlook and removing them from server.

IMAP (Internet Message Access Protocol)

Definition

IMAP allows users to **access and manage emails directly on the server.**

Characteristics

- Emails remain on server
- Syncs across **multiple devices**

Port

- 143** – IMAP
- 993** – Secure IMAP

Example

Checking the same Gmail inbox on mobile and laptop.

4. FTP (File Transfer Protocol)

Definition

FTP is used to **transfer files** between a client and a server.

Purpose

- Uploading/downloading files
- Website file management

Port Numbers

- 21 – Control
- 20 – Data

Example

Uploading website files to a web server using **FileZilla**

5. DNS (Domain Name System)

Definition

DNS translates **domain names** into **IP addresses**.

Purpose

- Makes internet user-friendly
- Converts names → numbers

How it works

www.google.com → 142.250.190.14

Port

- **53** (UDP mostly, TCP sometimes)

Example

Opening any website using a name instead of IP.

6. DHCP (Dynamic Host Configuration Protocol)

Definition

DHCP automatically assigns **IP address and network configuration** to devices.

Purpose

- Eliminates manual IP configuration
- Manages IP addresses dynamically

Information Provided

- IP address
- Subnet mask
- Gateway
- DNS server

Port Numbers

- **67** – Server
- **68** – Client

Example

Connecting a laptop to Wi-Fi and getting an IP automatically.

Web Links For Reference (Study Material)

`https://www.geeksforgeeks.org/computer-networks/computer-network-tutorials/`

NEXT----> Fundamentals of Programming Languages

- C/C++/Java Interview Questions Overview,,
- Variables, Data Types, and Memory,, Pointers and Storage, Classes,, Introduction to Object-Oriented
- Programming (OOP),, Compiler and Interpreter Basics,, Understanding Process Loading and Linking,,
- Techniques of Parameter Passing and Binding,, Storage Organization and Storage Classes,, Advanced
- OOP Concepts and Data Structures, Abstract Data Types (ADTs),, Data Structure Principles,,
- Advanced Programming Techniques,, Memory Handling in OOP Languages